TECNOLÓGICA DEL ORIENTE
INGENIERÍA DE SOFTWARE

PRUEBA Y CALIDAD DE SOFTWARE
UNIDAD 2 — ACTIVIDAD AA2

TALLER: AUTOMATIZACIÓN DE PRUEBAS CON JUNIT, SELENIUM Y POSTMAN

Pruebas unitarias, de API y de interfaz web, cobertura de código e integración continua

Estudiante: Carlos Arturo Rueda Calier
Modalidad: individual
Repositorio: github.com/CarlosDev88/calidad-software-u2-scripts-automatizados

Piedecuesta, Santander — 11 de noviembre de 2026

1. INTRODUCCIÓN Y OBJETIVO

Este informe documenta la automatización de pruebas funcionales de una aplicación web pequeña de registro de usuarios, formada por una API REST y un formulario web. El trabajo parte del proyecto de la actividad AA1, en el que se diseñaron scripts para la API y el formulario, y lo hace evolucionar hasta una suite completa: pruebas unitarias con JUnit 5 y Mockito, pruebas de API con TestRestTemplate y Postman, pruebas de interfaz con Selenium WebDriver, medición de cobertura con JaCoCo y ejecución automática en GitHub Actions.

El objetivo es demostrar, con código, comandos y resultados reales, que la suite verifica las reglas de la aplicación en los tres niveles de prueba, que se puede ejecutar con un solo comando en cualquier equipo y que el proceso seguido queda documentado paso a paso, con la justificación de cada decisión. Todas las cifras de este informe provienen de los archivos de la carpeta *evidencias/* del repositorio, generados el 11 de noviembre de 2026.

2. PUNTO DE PARTIDA Y ALCANCE

El repositorio se trabajó como una evolución del AA1 y no como un proyecto nuevo. Así el historial de git muestra cómo creció la suite y se reutilizan la aplicación, los datos de prueba y la infraestructura ya validados.

Aspecto	AA1 (punto de partida)	AA2 (este taller)
Niveles de prueba	API e interfaz	Unitaria, API e interfaz
Herramientas	JUnit 5, TestRestTemplate, Selenium, Postman	Se agregan Mockito, MockMvc y Bean Validation
Pruebas JUnit	21	50
Técnicas de diseño	Casos válidos e inválidos	Se agregan valores límite y particiones de equivalencia
Calidad del código de prueba	Sin medición	Cobertura JaCoCo con mínimo obligatorio del 80 %
Ejecución	Manual con Docker	Docker y además GitHub Actions en cada push
Documentación	README	README y guía paso a paso (GUIA_PASO_A_PASO.md)

3. APLICACIÓN BAJO PRUEBA

La aplicación está hecha con Spring Boot. Guarda los usuarios en memoria, lo que permite repetir las pruebas sin preparar una base de datos. Expone tres operaciones y un formulario *registro.html* que envía los datos a la API y muestra los errores debajo de cada campo.

Operación	Ruta	Respuestas
GET	/api/usuarios	200: lista de usuarios
GET	/api/usuarios/{id}	200: usuario; 404: no existe; 400: id no numérico
POST	/api/usuarios	201: creado; 400: datos inválidos o JSON mal formado; 409: email duplicado

Reglas de negocio, de las que salen los casos de prueba: el nombre es obligatorio y tiene de 3 a 60 caracteres; el email es obligatorio, debe tener formato válido y no puede repetirse; la edad es obligatoria y va de 18 a 100 años. Todos los errores se devuelven con un mismo formato JSON (estado, mensaje y error

por campo), definido en la clase *ManejadorErrores*.

4. HERRAMIENTAS, ENTORNO Y JUSTIFICACIÓN

Se siguió la pirámide de pruebas [10]: muchas pruebas unitarias rápidas en la base, menos pruebas de integración en el medio y pocas pruebas de interfaz en la parte alta, porque son las más lentas y las más sensibles a cambios en la pantalla.

Nivel	Herramienta y versión	Por qué se eligió
Unitaria	JUnit 5 y Mockito (incluidos en Spring Boot 3.3.5)	Estándar en Java [2]. Mockito permite probar el controlador sin el servicio real [3].
API	TestRestTemplate (Spring Boot 3.3.5)	Prueba la API con un servidor real en un puerto aleatorio, sin configuración extra [1].
API externa	Postman y Newman 6.x	Herramienta más usada para APIs; Newman la ejecuta desde consola y en CI [6].
Interfaz	Selenium WebDriver 4.25.0 y Chrome	Controla un navegador real, como lo haría un usuario [4, 5].
Cobertura	JaCoCo 0.8.12	Mide qué parte del código ejecutan las pruebas y puede frenar el build [7].
CI	GitHub Actions	Corre la suite en cada push sin depender de un equipo en particular [8].
Entorno	Docker Compose, Java 21, Maven 3.9.9	Versiones fijas: el resultado es el mismo en cualquier equipo [9].

ESTRUCTURA DEL ENTORNO

El archivo *docker-compose.yml* define cuatro contenedores. El de pruebas levanta la aplicación en un puerto aleatorio y Selenium la abre desde el contenedor de Chrome usando el nombre del servicio (*tests*) como dirección.

```
Equipo con Docker Desktop
 +-- chrome : selenium/standalone-chrome:4.25.0-20240922
 |            navegador + Selenium Grid (puerto 4444; vista en vivo en el 7900)
 +-- tests  : maven:3.9.9-eclipse-temurin-21
 |            mvn verify: levanta la app y corre las 50 pruebas JUnit + JaCoCo
 +-- app    : misma imagen; la app en el puerto 8080 (perfiles "demo" y "postman")
 +-- newman : postman/newman:6.1.3-alpine; corre la coleccion contra http://app:8080
```

La aplicación se construyó desde cero en lugar de probar un sitio público. Así las reglas y los datos están bajo control, las pruebas no fallan porque un tercero cambie su página y cada ejecución da el mismo resultado.

5. GUÍA PASO A PASO DEL PROCESO DE AUTOMATIZACIÓN

Esta sección resume el procedimiento seguido. El mismo contenido, con más detalle, está en *GUIA_PASO_A_PASO.md* del repositorio.

PASO 1. OBTENER EL PROYECTO Y VERIFICAR DOCKER

Solo se necesita Docker Desktop encendido; no hace falta instalar Java, Maven ni Chrome.

```
git clone https://github.com/CarlosDev88/calidad-software-u2-scripts-automatizados.git
cd calidad-software-u2-scripts-automatizados
docker version
docker compose version
```

PASO 2. DISEÑAR LOS CASOS A PARTIR DE LAS REGLAS

Por cada regla se definió el valor válido, el inválido y los límites. Se aplicaron dos técnicas de caja negra vistas en la Unidad 1 [11, 12]: **análisis de valores límite** (probar justo en el borde y justo afuera, donde suelen aparecer errores de < contra <=) y **particiones de equivalencia** (un representante por cada grupo de entradas que se comportan igual).

Regla	Valores límite probados	Particiones de equivalencia
Edad 18 a 100	17 (inválido), 18, 100 (válidos), 101 (inválido)	Nula; menor de edad; dentro del rango; mayor de 100
Nombre 3 a 60	2 (inválido), 3, 60 (válidos), 61 (inválido)	Vacío; corto; normal; largo
Email	—	Válido simple; válido con punto y +; sin @; sin dominio; sin usuario; dominio sin punto

PASO 3. ORGANIZAR LAS PRUEBAS POR NIVEL CON ETIQUETAS

Cada clase lleva la etiqueta *@Tag("unit")*, *@Tag("api")* o *@Tag("ui")*. Así se puede correr solo una parte de la suite: por ejemplo, las unitarias mientras se programa, porque tardan segundos.

```
src/test/java/co/edu/calidad/taller/
  unit/  UsuarioServiceTest, UsuarioRequestValidacionTest, UsuarioControllerTest   (29 pruebas)
  api/   UsuarioApiTest                                                           (13 pruebas)
  web/   RegistroFormularioTest + pages/RegistroPage + support/DriverFactory       (8 pruebas)
  support/ DatosPrueba, Evidencias, ReporteResultados                            (utilidades comunes)

mvn test -Dgroups=unit      mvn test -Dgroups=api      mvn test -Dgroups=ui
```

PASO 4. EJECUTAR TODA LA SUITE CON COBERTURA

Un solo comando compila, levanta la aplicación, corre las 50 pruebas JUnit, genera el reporte de cobertura y verifica que no baje del 80 %. Si una prueba falla o la cobertura no alcanza, el comando termina con error.

```
docker compose up --build --exit-code-from tests
# sin Docker, con Java 21 + Maven + Chrome instalados:
mvn verify
```

PASO 5. EJECUTAR LA COLECCIÓN POSTMAN

El perfil *postman* levanta la aplicación en el puerto 8080, espera a que responda y corre la colección con Newman. La colección también se puede importar en Postman y ejecutar con *Run collection*.

```
docker compose --profile postman run --rm newman
docker compose --profile demo --profile postman down     # apagar todo
```

PASO 6. REVISAR LAS EVIDENCIAS

Al terminar, la carpeta *evidencias/* contiene: *resultados-pruebas.txt* (cada prueba con OK o FALLO), *salida-consola.txt*, *surefire-reports/* (reportes JUnit con tiempos), *cobertura/*, *capturas/* (14 imágenes de Selenium) y *newman-resultados.xml*. Si una prueba de navegador falla, se guarda además una captura *FALLO_*.png* del momento exacto del error.

PASO 7. INTEGRACIÓN CONTINUA

Al subir los cambios a la rama *main*, GitHub Actions ejecuta el flujo *.github/workflows/pruebas.yml*, descrito en la sección 10.

6. PRUEBAS UNITARIAS CON JUNIT 5 Y MOCKITO

Las pruebas unitarias verifican una pieza aislada del código. Son 29 y se dividen en tres clases:

- **UsuarioServiceTest** (7 pruebas): crea el servicio con *new*, sin levantar Spring. Usa *@Nested* para agrupar por comportamiento ("Al crear un usuario", "Al consultar usuarios"). Verifica ids consecutivos, limpieza de espacios, email duplicado sin importar mayúsculas y excepción cuando el id no existe.
- **UsuarioRequestValidacionTest** (16 pruebas): aplica los valores límite y las particiones del paso 2 directamente sobre las reglas de validación, con *@ParameterizedTest* y *@CsvSource*.
- **UsuarioControllerTest** (6 pruebas): con *@WebMvcTest* se levanta solo la capa web y el servicio se reemplaza por un objeto simulado de Mockito. Así se comprueba que el controlador traduce bien cada situación a su código HTTP y que, si los datos son inválidos, nunca llama al servicio.

FRAGMENTOS REALES DEL PROYECTO

```
@ParameterizedTest(name = "edad {0} -> valida: {1}")
@CsvSource({ "17, false", "18, true", "100, true", "101, false" })
void limitesEdad(int edad, boolean esValida) {
    Set<String> errores = camposInvalidos(new UsuarioRequest("Ana Perez", "ana@correo.com", edad));
    assertThat(errores.contains("edad")).isEqualTo(!esValida);
}
```

```
@Test
@DisplayName("POST invalido responde 400 y NUNCA llama al servicio")
void crearInvalidoNoLlamaAlServicio() throws Exception {
    mockMvc.perform(post("/api/usuarios")
                .contentType(MediaType.APPLICATION_JSON).content(jsonInvalido))
           .andExpect(status().isBadRequest())
           .andExpect(jsonPath("$.errores.edad").value("La edad minima es 18"));
    verify(servicio, never()).crear(any());      // Mockito: el servicio no se uso
}
```

Clase	Casos principales y aserción	Ejecutadas	Resultado
UsuarioServiceTest	Ids 1 y 2; espacios eliminados; ANA@Correo.com duplicado lanza excepción; id 99 lanza UsuarioNoEncontradoException; limpiar() reinicia ids	7	7 OK
UsuarioRequestValidacionTest	Edad 17/18/100/101; nombre de 2/3/60/61 caracteres; 6 formatos de email; usuario válido sin errores; tres campos nulos	16	16 OK
UsuarioControllerTest	201 con cabecera Location; 400 sin llamar al servicio; 409 y 404 traducidos desde excepciones; lista; id "abc" responde 400	6	6 OK

```
● ● ●  Terminal — solo pruebas unitarias

$ mvn test -Dgroups=unit
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.066 s -- in unit.UsuarioControllerTest
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.265 s -- in unit.UsuarioRequestValidacionTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.018 s -- in unit.UsuarioServiceTest$Consultar
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in unit.UsuarioServiceTest$Crear
[INFO] Tests run: 29, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  6.879 s
```

Figura 1. Ejecución de solo las pruebas unitarias con la etiqueta "unit": 29 pruebas aprobadas; Maven terminó en 6,9 s incluyendo la compilación.

7. PRUEBAS DE API REST

TESTRESTTEMPLATE

UsuarioApiTest usa *@SpringBootTest(webEnvironment = RANDOM_PORT)*: levanta la aplicación real y hace peticiones HTTP verdaderas [1]. Antes de cada caso limpia los datos para que las pruebas sean independientes. Los siete casos de datos inválidos se resuelven con una sola prueba parametrizada.

Caso y entrada	Aserción principal	Resultado
POST válido (Ana, email único, 25)	201; id generado; datos iguales; cabecera Location	OK
GET lista con dos usuarios	200; lista de tamaño 2	OK
GET id existente / id 999	200 con el mismo usuario / 404 con el id en el mensaje	OK / OK
POST inválido x7: nombre vacío, "Al", email "ana-correo", email vacío, edad 15, 120 y nula	400; error en el campo correcto; no se guarda nada	7 OK
POST email repetido	409; error en email; queda un solo usuario	OK
POST con JSON roto	400; mensaje "mal formada"	OK

POSTMAN Y NEWMAN

La colección tiene siete peticiones en orden: crear (201), consultar por id (200), listar (200), email inválido (400), menor de edad (400), email duplicado (409) e id inexistente (404). La primera guarda el id y el email creados en variables que usan las siguientes.

A nivel de colección se define una prueba común a todas las peticiones: respuesta en menos de 2 segundos y en formato JSON [6].

Resultado: 7 peticiones y 29 aserciones, sin fallos; tiempo de respuesta promedio de 40 ms.

```
● ● ●  Terminal — Newman

$ newman run postman/usuarios.postman_collection.json --env-var baseUrl=http://localhost:8080

→ 06 - Crear usuario con email duplicado (409)
  POST http://localhost:8080/api/usuarios [409 Conflict, 322B, 19ms]
  ✓  Responde en menos de 2 segundos
  ✓  Responde en formato JSON
  ✓  Status 409 Conflict
  ✓  Indica que el email ya esta registrado

→ 07 - Obtener usuario inexistente (404)
  GET http://localhost:8080/api/usuarios/999999 [404 Not Found, 245B, 9ms]
  ✓  Responde en menos de 2 segundos
  ✓  Responde en formato JSON
  ✓  Status 404 Not Found
  ✓  El mensaje indica el id buscado

┌─────────────────────────┬──────────┬──────────┐
│                         │ executed │   failed │
├─────────────────────────┼──────────┼──────────┤
│              iterations │        1 │        0 │
├─────────────────────────┼──────────┼──────────┤
│                requests │        7 │        0 │
├─────────────────────────┼──────────┼──────────┤
│            test-scripts │       14 │        0 │
├─────────────────────────┼──────────┼──────────┤
│      prerequest-scripts │        1 │        0 │
├─────────────────────────┼──────────┼──────────┤
│              assertions │       29 │        0 │
├─────────────────────────┴──────────┴──────────┤
│ total run duration: 542ms                     │
├───────────────────────────────────────────────┤
│ total data received: 646B (approx)            │
├───────────────────────────────────────────────┤
│ average response time: 40ms [min: 6ms, max: 193ms, s.d.: 62ms] │
└───────────────────────────────────────────────┘
```

Figura 2. Final de la ejecución de Newman: peticiones 06 y 07 y resumen sin fallos.

8. PRUEBAS DEL FORMULARIO WEB CON SELENIUM WEBDRIVER

RegistroFormularioTest abre Chrome, llena el formulario y lee los mensajes que ve el usuario. Se aplicaron tres prácticas recomendadas por la documentación de Selenium [4, 5, 13]:

- **Page Object**: *RegistroPage* concentra los localizadores y las acciones (abrir, llenar, enviar, leer errores). Si cambia el HTML, se corrige en un solo lugar.
- **Esperas explícitas**: *WebDriverWait* de hasta 10 segundos espera a que aparezca el mensaje, en lugar de pausas fijas.
- **Evidencia automática**: cada caso guarda una captura con los datos de entrada y otra con el resultado; si falla, *CapturaEnFallo* guarda la pantalla antes de cerrar el navegador. *DriverFactory* usa el Chrome del contenedor o uno local sin cambiar las pruebas.

Caso y entrada	Aserción principal	Resultado
Registro válido (Ana, email único, 25)	Mensaje de éxito; campos limpios	OK
Email "ana-correo"	Error de formato bajo el campo email	OK
Nombre vacío / nombre "Al"	Error de obligatorio / de longitud	OK / OK
Edad 15 / edad 120	Error de edad mínima / máxima	OK / OK
Formulario vacío	Error en los tres campos	OK
Email repetido tras un registro	Error de email ya registrado	OK

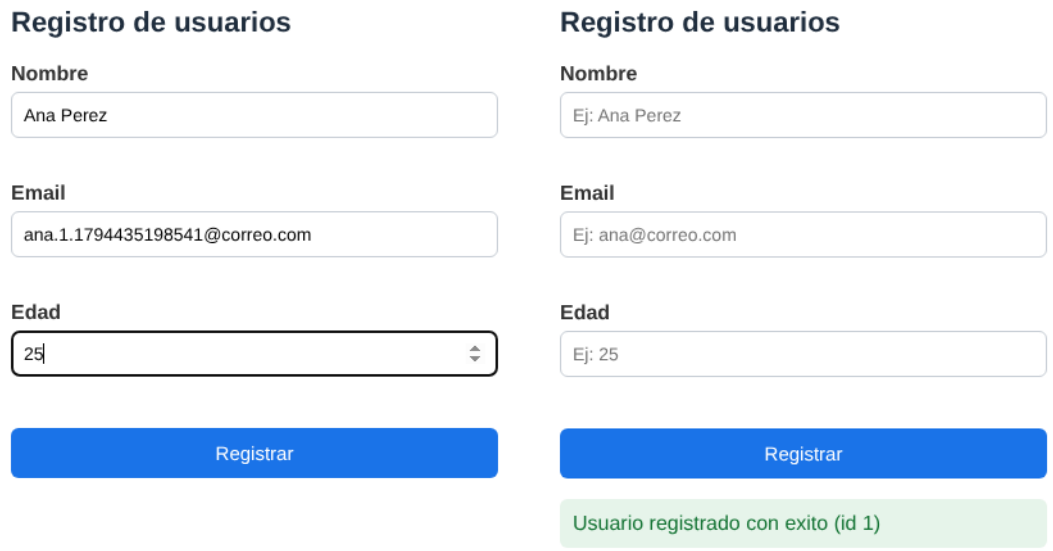


Figura 3. Registro válido: datos de entrada (izquierda) y resultado con mensaje de éxito y formulario limpio (derecha).

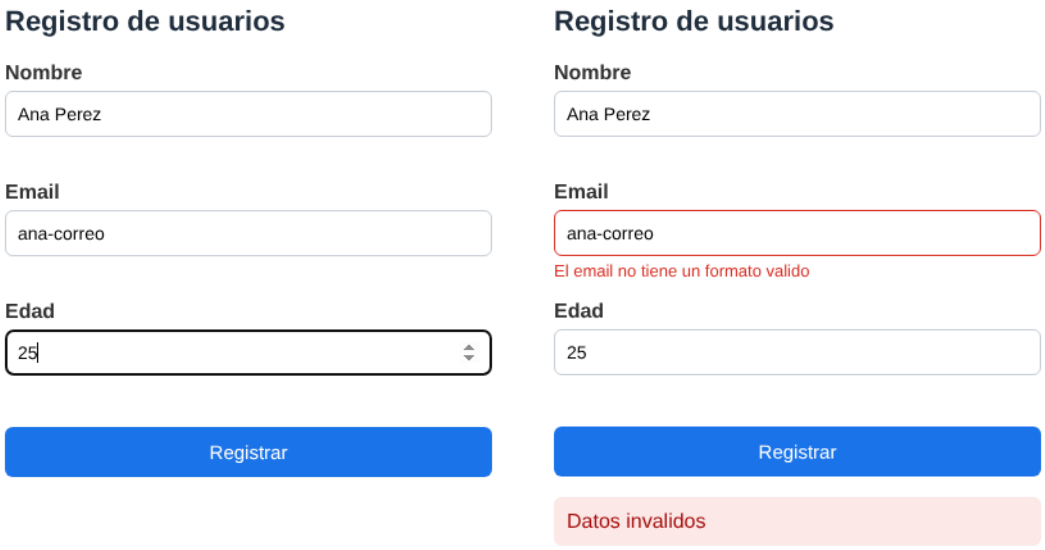


Figura 4. Email con formato inválido: datos de entrada y error mostrado bajo el campo.

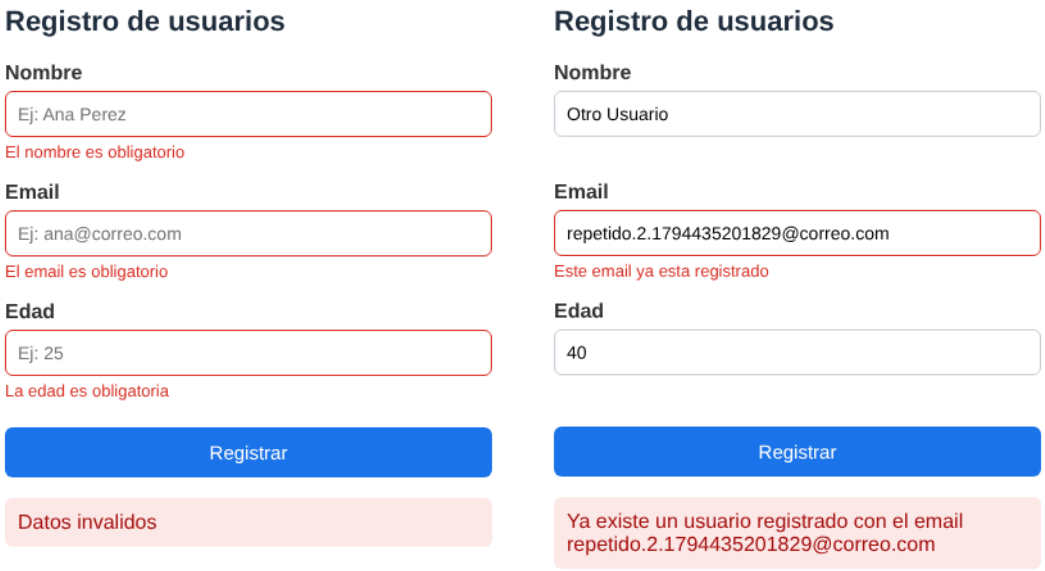


Figura 5. Formulario vacío con error en los tres campos (izquierda) y email duplicado (derecha).

9. COBERTURA Y EFICIENCIA DE LA SUITE

JaCoCo se configuró en el *pom.xml* para medir la cobertura durante las pruebas y para que *mvn verify* falle si la cobertura de líneas baja del 80 % [7]. Se excluyó solo la clase que arranca Spring, porque no tiene lógica. Resultado: 100 % de líneas (45 de 45), de instrucciones (256 de 256) y de ramas (14 de 14), en 8 clases.

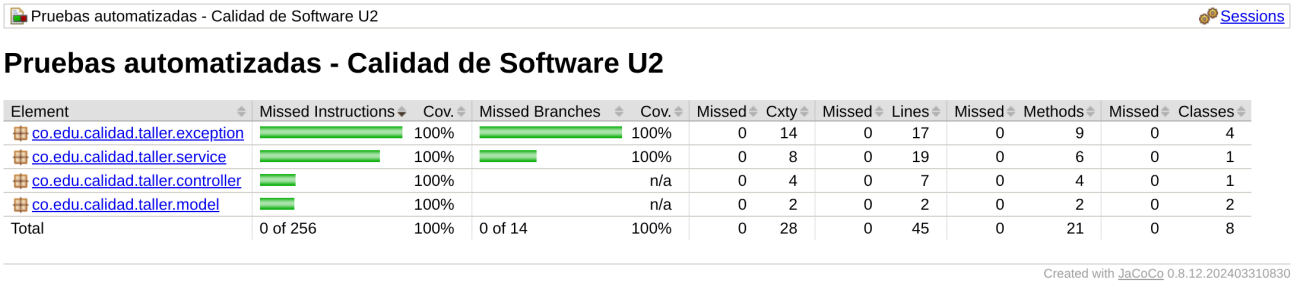


Figura 6. Reporte de JaCoCo: cobertura del 100 % en los cuatro paquetes de la aplicación.

Una cobertura del 100 % indica que todas las líneas se ejecutaron al menos una vez, no que el sistema esté libre de errores. Por eso se combina con aserciones concretas y con técnicas de diseño de casos. En una aplicación más grande, la meta razonable es mantener el mínimo exigido y revisar qué partes quedan sin probar.

La eficiencia se midió con los tiempos de los reportes JUnit. Los datos confirman la pirámide: las pruebas unitarias son las más numerosas y las más rápidas, mientras que las de navegador concentran casi todo el tiempo.

Nivel	Pruebas	Tiempo (s)	Tiempo por prueba
Unitarias (JUnit 5 + Mockito)	29	2,03	0,07 s
API (TestRestTemplate)	13	0,57	0,04 s
Interfaz (Selenium)	8	24,16	3,02 s
Total JUnit	50	26,76	—
Postman (Newman)	7 peticiones / 29 aserciones	0,54	40 ms promedio por respuesta

10. INTEGRACIÓN CONTINUA CON GITHUB ACTIONS

El archivo *.github/workflows/pruebas.yml* define dos trabajos que se ejecutan en cada push a *main*, en cada pull request o a mano desde la pestaña *Actions* [8]. El segundo solo corre si el primero pasa. Las evidencias quedan como archivos descargables de cada ejecución.

```
jobs:
  junit-selenium:                       # JUnit + Selenium + cobertura
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4          # Java 21 Temurin, cache de Maven
        with: { distribution: temurin, java-version: "21", cache: maven }
      - run: mvn -B verify                   # el runner trae Chrome; falla si algo falla
      - uses: actions/upload-artifact@v4     # evidencias, reportes JUnit y JaCoCo
  postman:
    needs: junit-selenium
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker compose --profile postman run --rm newman
```

Con esto la suite deja de depender de que alguien se acuerde de ejecutarla: cualquier cambio que rompa una regla de negocio o baje la cobertura queda marcado en rojo en GitHub antes de integrarse.

11. RESULTADOS CONSOLIDADOS

Fuente de evidencia	Ejecutadas	Aprobadas	Fallos / errores
JUnit unitarias	29	29	0 / 0
JUnit API (TestRestTemplate)	13	13	0 / 0
JUnit formulario (Selenium)	8	8	0 / 0
Total JUnit	50	50	0 / 0
Newman: peticiones	7	7	0 / 0
Newman: aserciones	29	29	0 / 0
Cobertura de líneas (mínimo 80 %)	45 líneas	45 cubiertas (100 %)	Regla cumplida

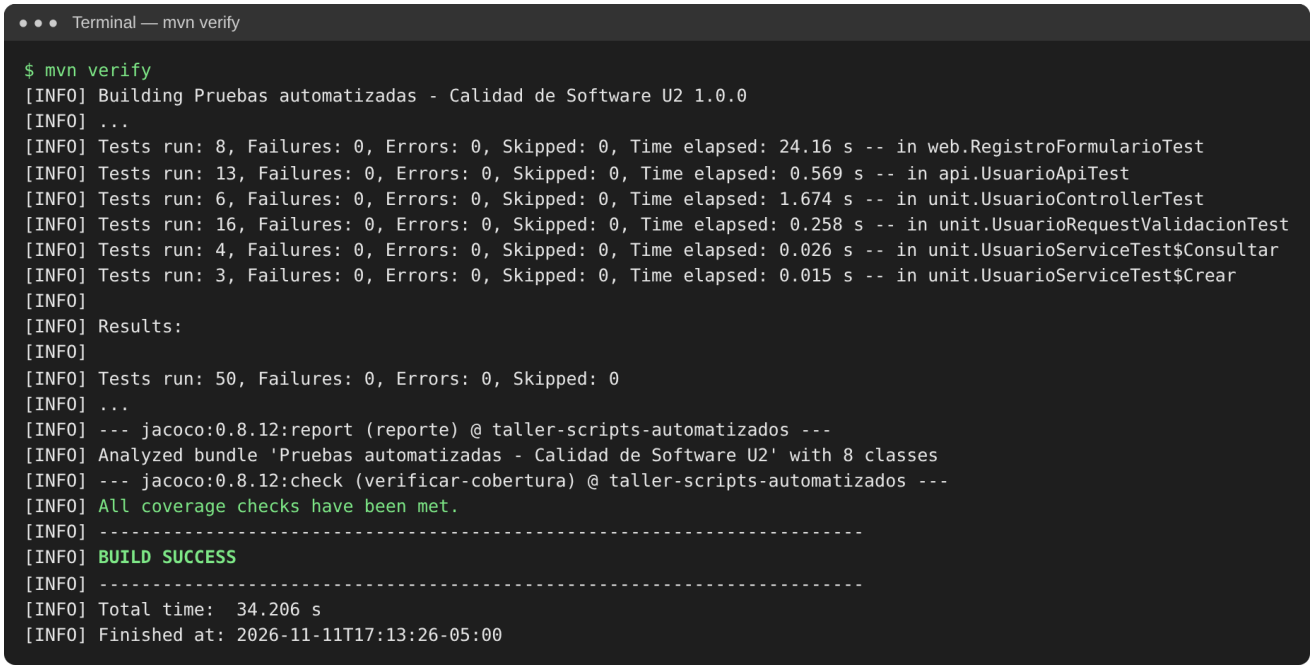


Figura 7. Ejecución completa con mvn verify: 50 pruebas sin fallos, cobertura cumplida y BUILD SUCCESS.

Las evidencias se generaron con *mvn verify* sobre Java 21 y Maven 3.9, con Chrome en modo headless, y con Newman desde la línea de comandos. El mismo proyecto se ejecuta con las versiones fijas de Docker Compose y en GitHub Actions, como se describe en las secciones 5 y 10.

12. ORGANIZACIÓN DEL TRABAJO

La actividad está planteada como taller grupal, pero se desarrolló de forma individual, por lo que no se reporta participación de otros integrantes. Para cubrir las tareas que en un equipo se repartirían, el trabajo se organizó por roles y se usó el repositorio de git como registro del avance:

Rol	Tareas realizadas
Analista de pruebas	Definición de reglas, diseño de casos con valores límite y particiones de equivalencia.
Automatizador	Pruebas unitarias, de API, de interfaz y colección Postman.
Responsable de entorno y CI	Docker Compose con versiones fijas, cobertura JaCoCo y flujo de GitHub Actions.
Documentador	Guía paso a paso, README, evidencias e informe.

13. REFLEXIÓN FINAL

Lo más valioso de este taller fue pasar de tener scripts sueltos a tener una suite que se ejecuta sola y que avisa cuando algo se rompe. Las pruebas unitarias me mostraron que la mayor parte de las reglas se pueden verificar en milisegundos, sin abrir un navegador; las de Selenium quedan para confirmar lo que realmente ve el usuario, que es más lento pero no se puede reemplazar. La tabla de tiempos de la sección 9 lo deja claro y explica por qué tiene sentido la pirámide de pruebas.

También aprendí a no confiar solo en el porcentaje de cobertura: llegar al 100 % fue fácil en una aplicación pequeña, pero lo que da valor son las aserciones y los casos en los límites, que es donde aparecieron las decisiones importantes (por ejemplo, que la edad 18 sea válida y la 17 no). Como limitación, la aplicación guarda los datos en memoria, así que la suite no evalúa persistencia ni concurrencia, y las pruebas automatizadas no sustituyen una revisión exploratoria. El siguiente paso natural sería agregar una base de datos real con pruebas de integración y medir el rendimiento de la API.

REFERENCIAS

[1] Spring. (2024). *Testing Spring Boot applications* (Spring Boot 3.3). https://docs.spring.io/spring-boot/3.3/reference/testing/spring-boot-applications.html

[2] JUnit Team. (s. f.). *JUnit 5 user guide*. https://junit.org/junit5/docs/current/user-guide/

[3] Mockito. (s. f.). *Mockito framework site*. https://site.mockito.org/

[4] Selenium Project. (s. f.). *Page object models*. https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/

[5] Selenium Project. (s. f.). *Waiting strategies*. https://www.selenium.dev/documentation/webdriver/waits/

[6] Postman. (s. f.). *Run collections using Newman CLI*. https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/

[7] Mountainminds GmbH & Co. KG. (s. f.). *JaCoCo: Java code coverage library – documentation*. https://www.jacoco.org/jacoco/trunk/doc/

[8] GitHub. (s. f.). *GitHub Actions documentation*. https://docs.github.com/en/actions

[9] Docker Inc. (s. f.). *Docker Compose*. https://docs.docker.com/compose/

[10] Fowler, M. (2012). *TestPyramid*. https://martinfowler.com/bliki/TestPyramid.html

[11] International Software Testing Qualifications Board. (2023). *Certified Tester Foundation Level syllabus* (versión 4.0). ISTQB. https://www.istqb.org/

[12] Myers, G. J., Sandler, C. y Badgett, T. (2011). *The art of software testing* (3.ª ed.). John Wiley & Sons.

[13] Fowler, M. (2013). *PageObject*. https://martinfowler.com/bliki/PageObject.html